

Enterprise Google Cloud Data Platform Blueprint

VOLUME 09

DevSecOps and CI/CD

Enterprise Source, Build, Supply-chain and Continuous Delivery Blueprint

Architecture Baseline 1.0 | 12 July 2026



Figure 0-1. Secure source-to-production flow across source, trusted build, immutable artifacts, policy gates, target-specific deployment and operational feedback.

Fresh generation - This package is generated from scratch. No artifact or state from the stalled Volume 9 execution is reused.

Document Control

Field	Value
Document	Volume 09 - DevSecOps and CI/CD
Series	Enterprise Google Cloud Data Platform Blueprint
Version	1.0
Status	Detailed reference architecture for adaptation and implementation
Date	12 July 2026
Owner	Enterprise DevSecOps Architecture
Approvers	CIO, CISO, CDO, Enterprise Architecture, Platform Engineering, Data Engineering, AI Platform, SRE, Change Management and Risk
Audience	Executives, architects, developers, data/ML engineers, platform engineers, security, SRE, audit and delivery teams
Review cycle	Quarterly and after material provider, supply-chain, regulation or incident change
Dependencies	Volumes 01-08, 10-12 and future Volume 13 Terraform implementation
Scope	Source control, builds, artifacts, IaC/data/application delivery, testing, policy, supply-chain, promotion, rollback, audit and release metrics

Guidance classes - Mandatory baseline controls require a time-bound approved exception. Recommended controls are enterprise defaults. Conditional controls apply when target, risk or regulation justifies them.

Table of Contents

1. Executive Summary
2. Inherited Context and Assumptions
3. DevSecOps Strategy
4. Operating Model and Responsibilities
5. Source-control Strategy
6. GitHub Architecture
7. Repository Structure and Monorepo versus Multi-repo
8. Branching, Versioning and Release Strategy
9. Target CI/CD Architecture
10. Cloud Build
11. GitHub Actions
12. Artifact Registry
13. Cloud Deploy
14. Terraform Pipelines
15. Application Pipelines
16. Dataform Pipelines
17. Dataflow Pipelines
18. Airflow Deployment
19. BigQuery and Schema Deployment
20. Policy-as-Code
21. Security and Supply-chain Scanning
22. Testing Strategy
23. Environment Promotion
24. Approval and Change Management
25. Rollback, Blue-green and Canary
26. Immutable Artifacts and SLSA Alignment
27. Auditability, Monitoring and Release Metrics
28. Service Assessments
29. Architecture Decision Records
30. Risks and Mitigations
31. Implementation Roadmap
32. Validation and Production Readiness
33. Dependencies, Decisions and Next Actions
34. References

1. Executive Summary

Volume 09 establishes a secure and auditable enterprise delivery system for infrastructure, applications, data pipelines, analytical models, schemas and policy. It preserves the inherited Zero Trust model by eliminating long-lived pipeline credentials, separating source, build, deploy and runtime identities, and requiring immutable artifacts and release evidence. It also preserves the SRE and DR model by treating verification and rollback as release gates rather than documentation tasks.

- GitHub Enterprise is the source-control and collaboration platform; protected repositories, rulesets, CODEOWNERS and environments implement governance.
- GitHub Actions provides repository-native orchestration. Workload Identity Federation exchanges GitHub OIDC identity for short-lived Google Cloud credentials.
- Cloud Build provides trusted Google Cloud builds, private network access, provenance and artifact publication.
- Artifact Registry is the artifact system of record with immutable digests/tags, vulnerability analysis and controlled cleanup.
- Cloud Deploy is used for supported application targets. Terraform, Dataform, Dataflow, Airflow and BigQuery use dedicated release lanes because their rollback and state semantics differ.
- Binary Authorization, Artifact Analysis and policy-as-code enforce supply-chain controls before and after deployment.



Figure 1-1. Enterprise DevSecOps architecture and control flow.

2. Inherited Context and Assumptions

The volume does not reopen platform, security, data, AI, FinOps, SRE or DR decisions. It applies them to software and data delivery.

Inherited dimension	Volume 09 application
Fortune 100 scale	Golden workflows and federated repositories must support thousands of pipelines and products without one shared bottleneck.
Regions	Build, artifact and target resources align to AMER, EMEA and APAC data/residency boundaries.
Environments	Development, integration, test, staging and production use separate projects, identities, data and approvals.
Security	Group-based access, WIF, VPC Service Controls, private networking, selective CMEK, no service-account keys.
Data products	Domain teams own logic, tests and outcomes; platform teams own paved roads and mandatory controls.
SRE	Production release verification uses SLOs, business/data metrics, runbooks and tested rollback.
FinOps	Build, artifact, scanning and verification costs are labeled and attributed to product/domain.
DR	Source, release manifests, artifacts and configurations are recoverable and alternative build/deploy paths are tested.

Confirmed requirement - Generate Volume 9 from scratch and do not resume or depend on the stalled execution.

3. DevSecOps Strategy

The strategy makes secure delivery the default path. Control depth is proportional to workload risk, but the same identity, artifact and evidence model applies across every release lane.

- Shift deterministic checks to developer and pull-request feedback; reserve independent approvals for changes with material risk.
- Build once and promote the same digest or commit-derived release through every environment.
- Use policy-as-code and standardized release manifests so evidence is machine-verifiable.
- Treat pipeline code, schemas, prompts, dashboards and monitoring configuration as governed production assets.
- Measure flow and stability together; do not optimize deployment frequency by weakening review, rollback or SLO gates.
- Continuously validate runtime conformance and dependency/artifact lifecycle after deployment.

Strategic outcome	Target evidence
Keyless delivery	100% external CI/CD access uses WIF/short-lived credentials
Artifact integrity	100% production artifacts are digest-addressed and linked to provenance
Automated assurance	Mandatory scans/policies run before merge/build/deploy
Safe promotion	Same artifact verified in lower environments and independently approved
Recoverability	Every production release records known-good rollback and service-specific procedure
Auditability	Commit, workflow, build, artifact, approval, deployment and telemetry are linked

4. Operating Model and Responsibilities

The operating model separates platform engineering, product delivery, security policy, SRE and change approval. Central teams provide paved roads; product teams retain accountability for functionality, data quality and business outcomes.

Role	Accountability
Enterprise DevSecOps platform	GitHub organization standards, golden workflows, build pools, artifact repositories, release manifest and support
Product/domain team	Source code, tests, contracts, release notes, service ownership and business acceptance
Security engineering	Policy library, vulnerability/secrets thresholds, Binary Authorization and exceptions
SRE	Operational readiness, deployment monitoring, canary criteria, incident and rollback practice
Change approver	Independent production decision according to risk class
Audit/risk	Control evidence, exception review and periodic assurance
FinOps	Build/artifact/scanning cost allocation and optimization

5. Source-control Strategy

Source control is the authoritative record for human-readable code, configuration, schemas, policies and deployment declarations. Generated artifacts and secrets are stored elsewhere.

- Enterprise SSO, MFA, least privilege and lifecycle automation govern organization membership.
- Repositories use visibility, fork and external-collaborator policies aligned to classification.
- Rulesets protect default and release branches; required checks cannot be bypassed by normal developers.
- CODEOWNERS assigns platform, security, data-steward and domain review by path.
- Repository templates establish issue, pull request, security, dependency and release conventions.
- GitHub audit events are exported to the enterprise SIEM and correlated with Google Cloud audit logs.

6. GitHub Architecture

GitHub Enterprise organizations are segmented by enterprise platform, shared capabilities and regulated business domains. GitHub is outside Google Cloud service perimeters, so access into Google Cloud is limited to OIDC federation and explicit APIs.

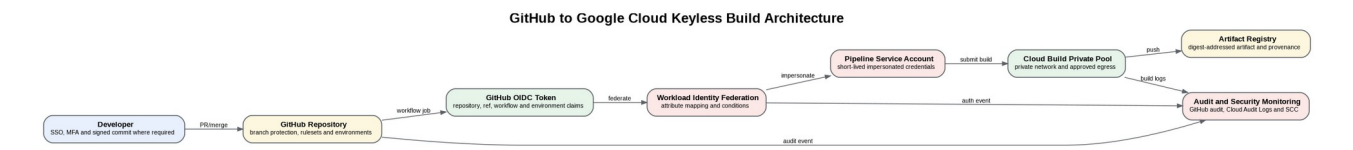


Figure 6-1. Keyless GitHub-to-Google Cloud build path using OIDC and Workload Identity Federation.

Control	Standard
Organization ownership	Enterprise-managed owners; no personal production ownership
Repository administration	Platform/product admin groups separate from code approvers
Actions	Allowlisted actions; pin third-party actions by full commit SHA in production
OIDC claims	Restrict organization, repository, ref, reusable workflow and environment
Environments	Protection rules for staging and production with named approvers
Secrets	Prefer OIDC/WIF; use GitHub secrets only for non-Google unavoidable credentials with rotation
Audit	Enterprise audit streaming, webhook integrity and retention

7. Repository Structure and Monorepo versus Multi-repo

The default is a federated multi-repository model aligned to product ownership and release boundaries. A monorepo is approved when teams share one ownership, dependency graph and release model and can control blast radius with path-based policies.

Repository type	Contents	Owner	Release boundary
platform-foundation	Root Terraform and organization policy	Central platform	foundation wave
golden-modules	Reusable IaC/build/deploy modules	Platform engineering	semantic module version
policy-library	OPA and security policy	Security architecture	policy version
domain-data-product	Dataform, pipelines, schemas, contracts	Domain product	data-product release
application-service	application and manifests	Product team	service release

Anti-pattern - One enterprise monorepo that creates shared admin rights, coupled release timing and an unbounded blast radius.

8. Branching, Versioning and Release Strategy

Trunk-based development with short-lived branches is the default. Pull requests are small, independently reviewable and protected by required checks. Release branches are reserved for supported maintenance or regulated release trains, not routine integration.

- Use semantic versioning for reusable modules, libraries and externally consumed contracts.
- Use commit SHA and artifact digest as the immutable technical identity of every build.
- Use calendar or domain release identifiers only as human-readable aliases, never as the sole artifact identity.
- Tag releases after successful build and evidence creation; protected tags cannot be moved.
- Release notes include change, risk, compatibility, migration, rollback, owner and known limitations.
- Deprecation policies define support windows and migration deadlines for modules, schemas and APIs.

9. Target CI/CD Architecture

The architecture separates repository checks, trusted builds, artifact custody, policy enforcement and target-specific deployment. This avoids granting one tool broad rights across source, production and runtime.



Figure 9-1. Supply-chain controls from source verification to continuous validation.

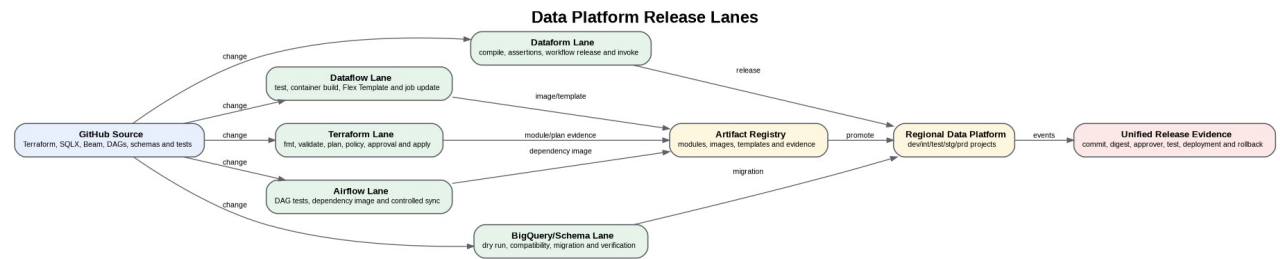


Figure 9-2. Dedicated deployment lanes for Terraform, Dataform, Dataflow, Airflow and BigQuery/schema assets.

10. Cloud Build

Cloud Build is the trusted build plane for containers, Dataflow templates and other Google Cloud artifacts. Private pools are used when builds require private resources, custom networking or controlled egress.

- Use dedicated build projects and service accounts by trust/region rather than default broad project identities.
- Pin builder images by digest where practical and maintain an approved builder catalog.
- Generate build provenance and link it to artifact digest and release manifest.
- Use Secret Manager only for unavoidable third-party credentials; prefer WIF and service identity.
- Store test results, SBOM and scan evidence in a protected evidence store.
- Apply timeouts, machine sizing, disk and concurrency limits; alert on queued or repeatedly failed builds.

```
# Purpose: trusted application build with tests, vulnerability evidence and immutable image publication.
substitutions:
  _REGION: us-central1
  _REPOSITORY: app-images
  _IMAGE: customer-api
steps:
- id: unit-and-static-tests
  name: python:3.12-slim
  entrypoint: bash
  args: ['-ceu', 'pip install -r requirements-dev.txt && ruff check . && pytest -q --junitxml=/workspace/test-
results.xml']
- id: build-image
  name: gcr.io/cloud-builders/docker
  args: ['build', '--pull', '-t', '${_REGION}-docker.pkg.dev/$PROJECT_ID/${_REPOSITORY}/${_IMAGE}:$COMMIT_SHA', '.']
- id: push-image
  name: gcr.io/cloud-builders/docker
  args: ['push', '${_REGION}-docker.pkg.dev/$PROJECT_ID/${_REPOSITORY}/${_IMAGE}:$COMMIT_SHA']
- id: inspect-vulnerabilities
  name: gcr.io/google.com/cloudsdktool/cloud-sdk:slim
  entrypoint: bash
  args: ['-ceu', 'gcloud artifacts docker images describe ${_REGION}-docker.pkg.dev/$PROJECT_ID/${_REPOSITORY}/${_
IMAGE}:$COMMIT_SHA --show-package-vulnerability --format=json > /workspace/vulnerability.json']
- id: policy-gate
  name: openpolicyagent/conftest:v0.57.0
  args: ['test', '/workspace/vulnerability.json', '--policy', 'policy', '--all-namespaces']
images: [`${_REGION}-docker.pkg.dev/$PROJECT_ID/${_REPOSITORY}/${_IMAGE}:$COMMIT_SHA`]
options:
  logging: CLOUD_LOGGING_ONLY
  pool: {name: 'projects/$PROJECT_ID/locations/${_REGION}/workerPools/private-build'}
timeout: 1800s
```

11. GitHub Actions

GitHub Actions executes repository-native checks and submits trusted work to Google Cloud using Workload Identity Federation. It is not granted permanent production credentials.

- Use reusable workflows maintained by the platform team and invoked by product repositories.
- Restrict id-token permission to jobs that need federation; default all other permissions to read or none.
- Use protected environments and independent approval for production jobs.
- Disable credential persistence in checkout and avoid untrusted pull-request code with privileged tokens.
- Use concurrency groups to prevent conflicting releases and cancel obsolete validation runs.
- Pin actions and review transitive scripts; export audit and workflow telemetry.

```
name: build-and-release
on:
  pull_request:
  push: {branches: [main]}
permissions:
  contents: read
  id-token: write
  security-events: write
concurrency:
  group: ${ github.workflow }-${ github.ref }
  cancel-in-progress: true
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: {persist-credentials: false}
      - uses: actions/setup-python@v5
        with: {python-version: '3.12'}
      - run: pip install -r requirements-dev.txt && ruff check . && pytest -q
  submit-cloud-build:
    if: github.ref == 'refs/heads/main'
    needs: validate
    runs-on: ubuntu-latest
    environment: integration
    steps:
      - uses: actions/checkout@v4
        with: {persist-credentials: false}
      - id: auth
        uses: google-github-actions/auth@v2
        with:
          workload_identity_provider: ${ vars.WIF_PROVIDER }
          service_account: ${ vars.BUILD_SUBMITTER_SA }
          token_format: access_token
      - uses: google-github-actions/setup-gcloud@v2
      - run: gcloud builds submit --project="${ vars.BUILD_PROJECT }" --region="${ vars.REGION }" --
config=cloudbuild.yaml --substitutions=COMMIT_SHA=${ github.sha } .
```

12. Artifact Registry

Artifact Registry is the system of record for production images, packages and reusable modules. Repositories are segmented by format, region, trust and lifecycle.

- Production Docker repositories enable immutable tags and require digest-pinned deployment.
- Automatic vulnerability scanning is enabled where required and supported; findings feed release policy and SCC.
- Cleanup policies are tested in dry-run and preserve tagged approved rollback artifacts.
- Remote/virtual repositories may reduce uncontrolled internet dependency access after security review.
- CMEK, IAM, VPC Service Controls and regional placement follow artifact classification.
- Artifact writers cannot deploy to production; deployers cannot overwrite artifacts.

13. Cloud Deploy

Cloud Deploy is used for supported application delivery targets such as Cloud Run and GKE, including standard and canary strategies, approvals and post-deploy verification. It is not forced onto Terraform or data assets with different state semantics.

- Create one release from a digest-pinned image and promote it through environment targets.
- Use target-specific delivery service accounts and protected production approvals.
- Configure verification tasks for health and critical user journeys.
- Use canary percentages and automatic/manual progression based on workload risk.
- Integrate Binary Authorization separately; Cloud Deploy and Binary Authorization are complementary controls.
- Record release, rollout, rendered manifest, deploy/verify logs and final state.

```
apiVersion: deploy.cloud.google.com/v1
kind: DeliveryPipeline
metadata: {name: customer-api}
description: Regional customer API delivery pipeline
serialPipeline:
  stages:
    - targetId: integration
      profiles: [integration]
    - targetId: staging
      profiles: [staging]
    - targetId: production
      profiles: [production]
  strategy:
    canary:
      runtimeConfig: {cloudRun: {automaticTrafficControl: true}}
      canaryDeployment: {percentages: [10, 25, 50], verify: true}
---
apiVersion: deploy.cloud.google.com/v1
kind: Target
metadata: {name: integration}
run: {location: projects/egcdp-app-int/locations/us-central1}
---
apiVersion: deploy.cloud.google.com/v1
kind: Target
metadata: {name: staging}
requireApproval: true
run: {location: projects/egcdp-app-stg/locations/us-central1}
---
apiVersion: deploy.cloud.google.com/v1
kind: Target
metadata: {name: production}
requireApproval: true
run: {location: projects/egcdp-app-prd/locations/us-central1}
```


14. Terraform Pipelines

Terraform delivery separates validation, plan, approval and apply. The apply identity is more privileged than the plan identity and is available only through protected production environments.

- Run fmt, validate, provider lock verification, static/security analysis and policy tests on every pull request.
- Use remote state with encryption, restricted access, locking and state-version recovery.
- Review human-readable plan and machine-readable JSON policy results.
- Apply a saved plan only while commit, variables, provider lock and state serial remain valid; otherwise re-plan.
- Require independent approval for production and elevated approval for IAM, network, KMS, perimeter and destructive changes.
- Detect drift continuously; emergency console changes are reconciled to code.

```
name: terraform-plan-apply
on:
  pull_request: {paths: ['infra/**']}
  push: {branches: [main], paths: ['infra/**']}
permissions: {contents: read, id-token: write, pull-requests: write}
jobs:
  plan:
    runs-on: ubuntu-latest
    environment: terraform-plan
    steps:
      - uses: actions/checkout@v4
        with: {persist-credentials: false}
      - uses: hashicorp/setup-terraform@v3
        with: {terraform_version: '1.9.8'}
      - uses: google-github-actions/auth@v2
        with:
          workload_identity_provider: ${vars.WIF_PROVIDER}
          service_account: ${vars.TERRAFORM_PLAN_SA}
      - run: terraform -chdir=infra fmt -check -recursive
      - run: terraform -chdir=infra init -input=false
      - run: terraform -chdir=infra validate
      - run: terraform -chdir=infra plan -input=false -lock-timeout=5m -out=tfplan
      - run: terraform -chdir=infra show -json tfplan > tfplan.json
      - run: conftest test infra/tfplan.json --policy policy/terraform
      - uses: actions/upload-artifact@v4
        with: {name: terraform-plan, path: 'infra/tfplan*', retention-days: 7}
  apply:
    if: github.ref == 'refs/heads/main'
    needs: plan
    runs-on: ubuntu-latest
    environment: production-apply
    steps:
      - uses: actions/checkout@v4
        with: {persist-credentials: false}
      - uses: hashicorp/setup-terraform@v3
        with: {terraform_version: '1.9.8'}
      - uses: google-github-actions/auth@v2
        with:
          workload_identity_provider: ${vars.WIF_PROVIDER}
          service_account: ${vars.TERRAFORM_APPLY_SA}
      - run: terraform -chdir=infra init -input=false
      - run: terraform -chdir=infra apply -input=false -lock-timeout=5m -auto-approve
```

15. Application Pipelines

Application pipelines validate code, package an immutable artifact, scan it, deploy progressively and verify both technical and business health.

Stage	Application control
Pull request	unit, SAST, dependency, contract and policy tests
Build	reproducible/pinned build, SBOM, provenance and scan
Integration	managed dependencies and identity/network tests
Staging	production-like configuration, load, resilience and rollback rehearsal
Production	approval, canary/blue-green, SLO/business verification and rollback
Post-release	release annotation, metrics, incident linkage and artifact lifecycle

16. Dataform Pipelines

Dataform pipelines compile an immutable repository commit, execute assertions, promote release/workflow configuration and verify BigQuery outputs. SQL development and runtime identities are separated.

- Compile on pull request and production release; fail on graph or SQL compilation errors.
- Run assertions and data-contract tests before dependent assets are certified.
- Use environment-specific release/workflow configuration, not environment branches with divergent SQL.
- Capture compilation result, workflow invocation, BigQuery job IDs, assertion results and commit.
- Rollback uses the last approved commit/configuration and a governed data correction strategy.

17. Dataflow Pipelines

Dataflow pipelines build versioned Flex Template artifacts from tested Apache Beam code. Streaming deployments distinguish compatible update from drain, snapshot, restart and replay.

- Run unit and Beam test-pipeline tests before the image build.
- Build and scan an immutable container; publish a versioned Flex Template spec.
- Deploy with dedicated runtime identity, network, KMS and environment parameters.
- Test state/timer/schema compatibility before streaming update.
- Record template path, image digest, job ID, source offsets/snapshot and reconciliation evidence.
- Rollback is not assumed to be a simple image change; use the Volume 4 and Volume 12 recovery semantics.

18. Airflow Deployment

Managed Airflow releases consist of DAGs, dependencies, plugins/configuration and environment image/version changes, each with distinct risk and rollback.

- Parse and unit-test DAGs without contacting production services.
- Keep DAG code thin; transformation runs in managed execution engines.
- Synchronize a versioned DAG bundle through a controlled deployment identity.
- Pin provider/dependency versions and test in a non-production environment before image upgrade.
- Monitor import errors, scheduler health, queue, task failures and backfill concurrency.
- Rollback the DAG bundle or environment configuration separately and verify scheduler state.

19. BigQuery and Schema Deployment

BigQuery releases include SQL objects, routines, views, authorized views, policy tags, row policies and schema migrations. Data compatibility and recovery are first-class because infrastructure-style rollback is often insufficient.

- Use dry-run query validation, maximum bytes guardrails and static SQL checks.
- Classify migration as additive, compatible, deprecating or destructive.
- Use expand-and-contract and compatibility views for consumer-safe evolution.
- Snapshot or preserve time-travel/restore path before destructive Tier 0/1 changes.
- Execute reconciliation, quality and performance checks before certification.
- Record BigQuery job IDs, migration checksum, affected contracts and recovery decision.

20. Policy-as-Code

Policy-as-code converts mandatory architecture and security controls into versioned tests or enforcement. Policies have owners, tests, severity, exception process and staged rollout.

- Run local/PR policy tests for fast feedback and repeat them at trusted build/deploy boundaries.
- Use audit/dry-run before enforcing new broad organization, admission or cleanup policy.
- Keep policy input adapters stable and test provider/scanner schema changes.
- Separate mandatory deny policies from advisory recommendations.
- Exceptions require risk owner, rationale, compensating controls, scope and expiry.
- Measure false-positive, bypass and remediation rates.

```
package terraform.egcdp
import rego.v1

deny contains msg if {
  some rc in input.resource_changes
  rc.type == "google_service_account_key"
  msg := sprintf("User-managed service account keys are prohibited: %s", [rc.address])
}

deny contains msg if {
  some rc in input.resource_changes
  rc.type == "google_artifact_registry_repository"
  rc.change.after.format == "DOCKER"
  not rc.change.after.docker_config[0].immutable_tags
  msg := sprintf("Docker repository must enable immutable tags: %s", [rc.address])
}
```

21. Security and Supply-chain Scanning

Scanning is layered because no single tool covers source, secrets, dependencies, containers, infrastructure and runtime. Results are normalized into severity and ownership rather than simply accumulated.

Scan	Release decision
Secret	Any valid credential blocks merge; rotate and investigate
SAST	Critical/high exploitable finding blocks according to risk policy
Dependency/license	Block prohibited license and critical vulnerable dependency without approved mitigation
Container	Artifact Analysis and approved scanner produce artifact-linked findings
Terraform/IaC	Block public exposure, keys, broad IAM, missing encryption/perimeter controls
DAST/API	Block exploitable high-severity issue in target environment
Binary Authorization	Deny artifact lacking required attestations
Continuous validation	Open incident/remediation for running policy violation

22. Testing Strategy

Testing follows a pyramid but includes managed integration, data correctness, performance, security and recovery because pipeline success is not equivalent to business correctness.

- Unit tests isolate deterministic business and transformation logic.
- Contract tests validate API, event, schema and data-product compatibility.
- Integration tests use real managed services and synthetic/masked data in isolated projects.
- Data and quality tests validate completeness, reconciliation, freshness and business rules.
- Performance tests include volume, concurrency, skew, quotas and unit cost.
- Security tests cover identity, network, perimeter, secrets, malicious input and policy bypass.
- Resilience and rollback tests prove dependency failure, partial release and recovery.

Stage	Tests	Failure effect	Evidence
pre-commit	format, lint, local unit, secret	block commit/review	local/CI status
pull request	unit, SAST, dependency, IaC, policy, contract	block merge	checks and review
build	container/image, SBOM, provenance, vulnerability	block publication/attestation	build evidence
integration	managed service and identity/network	block promotion	run IDs/logs
test	data quality, performance, resilience, DAST	block staging	reports
staging	release verification and rollback rehearsal	block production approval	ORR record
production	smoke, canary analysis, SLO/business KPI	pause or rollback	rollout and telemetry

23. Environment Promotion

Environment promotion moves the same immutable release bundle through development, integration, test, staging and production. Configuration is versioned separately but must remain compatible with the artifact.



Figure 23-1. Environment promotion, independent approval and production verification.

Environment	Purpose	Data	Approval	Mandatory gates	Promotion
development	developer feedback	synthetic or masked	developer	lint, unit, secret, policy	branch/PR artifact
integration	managed service integration	synthetic representative	team lead	contract, integration, IAM/network	same digest
test	quality and scale	masked subset	QA/product	data, performance, resilience, security	same digest
staging	release rehearsal	production-like masked	SRE/security as risk requires	ORR, smoke, rollback, capacity	same digest
production	business service	production	independent approver	change, SLO, canary/blue-green, audit	same digest only

24. Approval and Change Management

Approval is risk-based and independent. Automated gates provide evidence, while human approvers decide changes involving business risk, destructive data effects, privileged control planes or regulated outcomes.

- The developer who authors a production change cannot be the sole production approver.
- Approval displays artifact digest, plan/diff, tests, security result, SLO impact, rollback ID and change record.
- Approval expires when artifact, configuration, target, state serial or risk evidence changes.
- Standard changes may use pre-authorized templates; exceptions and high-risk changes require additional review.
- Emergency changes use PAM/break-glass, incident/change linkage and mandatory post-implementation review.

25. Rollback, Blue-green and Canary

Deployment strategy follows workload semantics. Stateless application releases can use standard, blue-green or canary rollouts. Infrastructure, schemas, stateful streaming and data corrections require service-specific recovery designs.

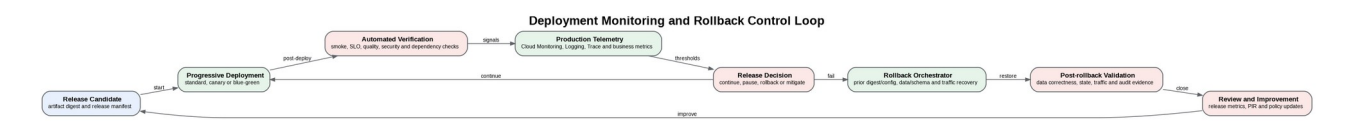


Figure 25-1. Verification, monitoring, rollback, reconciliation and review control loop.

Strategy	Best fit	Required guardrails
Standard	low-risk or internally verified target	verification, prior release and rapid rollback
Canary	user traffic can be safely segmented	representative cohort, technical/business SLOs, automatic pause
Blue-green	parallel full environment and traffic switch are feasible	state/data compatibility, capacity and cleanup
Forward-fix	additive data/schema defect with safe compatibility	bounded correction and consumer validation
Snapshot/restore or replay	data/state corruption or incompatible pipeline change	RPO/RTO, reconciliation and approval

26. Immutable Artifacts and SLSA Alignment

Production releases promote immutable artifacts and verifiable evidence. The program uses SLSA as a supply-chain assurance framework rather than claiming a level without evidence.

- Generate provenance identifying builder, invocation, source and dependencies.
- Use trusted isolated builders and protect build configuration from unauthorized modification.
- Store artifacts by digest and prevent tag reassignment for production repositories.
- Validate provenance and required attestations before deployment.
- Protect and retain release evidence according to audit and recovery policy.
- Assess SLSA specification revisions and document the achieved build-track controls per artifact class.

27. Auditability, Monitoring and Release Metrics

Release evidence links GitHub identity and commit to workflow, federated principal, Cloud Build, artifact digest, scans, attestations, approval, target rollout and production telemetry. This chain enables audit, incident investigation and reliable rollback.

- Annotate deployments in monitoring and incident systems with release ID and artifact digest.
- Alert on failed production rollout, policy/admission denial, abnormal approval or compromised credential/artifact finding.
- Measure DORA-style flow/stability metrics plus artifact reuse, policy escapes, approval latency and rollback exercises.
- Retain evidence by workload classification; avoid secrets, source payloads or sensitive production data in release logs.
- Review release metrics monthly and supply-chain controls quarterly.

Metric	Definition	Target use	Owner
deployment frequency	successful production deployments per period	flow/adoption	DevSecOps
lead time for change	merge to verified production	flow	Product team
change failure rate	deployments causing rollback/incident/hotfix	stability	SRE/Product
failed deployment recovery time	failure detection to validated recovery	resilience	SRE
policy escape rate	production violations not caught pre-deploy	assurance	Security
artifact reuse rate	production releases promoting same tested digest	integrity	DevSecOps
approval latency	ready-for-approval to decision	governance efficiency	Change management
rollback exercise success	successful tested rollbacks / scheduled	readiness	SRE

28. Google Cloud Service Assessments

Each selected service maps to a delivery or security requirement. Current quotas, regions, IAM, CMEK, VPC Service Controls, launch stage and API syntax must be validated during implementation.

28.1 Cloud Build

Assessment field	Enterprise guidance
Service name	Cloud Build
Purpose	Managed build and test execution
Enterprise architecture role	Trusted Google Cloud build plane for containers, templates and provenance
Business capabilities supported	Secure software and data delivery, controlled change, auditability, platform engineering and regulated release evidence.
Technical capabilities	Managed build and test execution integrated with IAM, logging, monitoring and Google Cloud resource controls.
Benefits	Managed integration, reduced undifferentiated operations, standard evidence and alignment with the inherited Google Cloud platform.
Limitations	Features, targets, quotas, locations, integrations and launch stages vary; some data assets require dedicated deployment lanes rather than application CD.
Service quotas	Validate project, regional, API, concurrency, storage and throughput quotas before scale rollout.
Regional availability considerations	Place regional resources such as private pools and repositories with the approved delivery/data plane; validate cross-region access and residency.
High-availability characteristics	Managed control planes reduce infrastructure operations; delivery workflows must tolerate API/transient failures and preserve source/artifacts for replay.
Disaster-recovery considerations	Back up source/configuration, replicate or rebuild artifact repositories where required, preserve provenance and test alternate build/deploy paths.
Security model	IAM, workload identity, protected repositories/environments, least privilege, private network access and immutable audit evidence.
IAM requirements	Separate source admin, workflow identity, build identity, artifact writer, deployer, approver and runtime identities.
Network requirements	Private pools or approved runners, Private Google Access/PSC where supported and controlled egress for dependencies.
VPC Service Controls considerations	Validate service support and ingress/egress dependencies; GitHub is outside the perimeter and must use WIF and narrowly authorized APIs.
Encryption capabilities	Google-managed encryption by default and TLS in transit; protect evidence, artifacts and configuration.
CMEK support	Use where required and supported, especially repositories/build resources; validate resource/location and service-agent permissions.
Logging	Enable audit, workflow, build, artifact and rollout logs; exclude secrets and sensitive payloads.
Monitoring	Track workflow/build/rollout success, duration, queue, policy failures, artifact age, vulnerabilities and release outcomes.
Alerting	Page only for production-impacting or supply-chain security events; route failed builds and policy trends to owners.
Cost drivers	Workflow minutes/runners, private build workers, storage, scanning, network, verification jobs and logging.
Cost-optimization techniques	Cache safely, right-size private pools, avoid duplicate rebuilds, apply cleanup policies and limit noisy logs without weakening evidence.
Operational responsibilities	Platform DevSecOps owns golden paths; product teams own code/tests and release outcomes; security owns policy; SRE owns production rollout/rollback practice.
Integration points	GitHub, IAM/WIF, Cloud Build, Artifact Registry, Cloud Deploy, Binary Authorization, Security Command Center, Monitoring, Terraform and target services.
Alternatives	GitLab/Jenkins/Azure DevOps, third-party registries/deployment tools or custom pipelines when approved requirements are unmet.
Selection criteria	Source governance, identity, network, artifact type, target support, regulated evidence, progressive delivery, operability and cost.
Best practices	Keyless identity, immutable digests, pinned dependencies, isolated builds, policy gates, same-artifact promotion, independent approval and tested rollback.
Reference architecture	GitHub -> PR checks -> WIF -> Cloud Build -> Artifact Registry -> policy/attestation -> target-specific deployment -> monitoring/rollback.
Sample configuration	See examples folders; validate exact action versions, API schema and provider arguments before production use.
Common pitfalls	Long-lived keys, mutable tags, unpinned actions, broad service accounts, rebuild per environment, unmanaged console deployment and absent rollback evidence.
Anti-patterns	CI identity is project owner; production deployment from developer laptop; secret in repository; latest tag; approval by code author only.
Production-readiness checklist	Repository controls; WIF; IAM separation; private build path; scans; immutable artifact; provenance; target approval; verification; rollback; metrics and retention.

28.2 GitHub Actions

Assessment field	Enterprise guidance
Service name	GitHub Actions
Purpose	Repository-native workflow automation
Enterprise architecture role	Source-control validation and orchestration with OIDC federation into Google Cloud
Business capabilities supported	Secure software and data delivery, controlled change, auditability, platform engineering and regulated release evidence.
Technical capabilities	Repository-native workflow automation integrated with IAM, logging, monitoring and Google Cloud resource controls.
Benefits	Managed integration, reduced undifferentiated operations, standard evidence and alignment with the inherited Google Cloud platform.
Limitations	Features, targets, quotas, locations, integrations and launch stages vary; some data assets require dedicated deployment lanes rather than application CD.
Service quotas	Validate project, regional, API, concurrency, storage and throughput quotas before scale rollout.
Regional availability considerations	Place regional resources such as private pools and repositories with the approved delivery/data plane; validate cross-region access and residency.
High-availability characteristics	Managed control planes reduce infrastructure operations; delivery workflows must tolerate API/transient failures and preserve source/artifacts for replay.
Disaster-recovery considerations	Back up source/configuration, replicate or rebuild artifact repositories where required, preserve provenance and test alternate build/deploy paths.
Security model	IAM, workload identity, protected repositories/environments, least privilege, private network access and immutable audit evidence.
IAM requirements	Separate source admin, workflow identity, build identity, artifact writer, deployer, approver and runtime identities.
Network requirements	Private pools or approved runners, Private Google Access/PSC where supported and controlled egress for dependencies.
VPC Service Controls considerations	Validate service support and ingress/egress dependencies; GitHub is outside the perimeter and must use WIF and narrowly authorized APIs.
Encryption capabilities	Google-managed encryption by default and TLS in transit; protect evidence, artifacts and configuration.
CMEK support	Use where required and supported, especially repositories/build resources; validate resource/location and service-agent permissions.
Logging	Enable audit, workflow, build, artifact and rollout logs; exclude secrets and sensitive payloads.
Monitoring	Track workflow/build/rollout success, duration, queue, policy failures, artifact age, vulnerabilities and release outcomes.
Alerting	Page only for production-impacting or supply-chain security events; route failed builds and policy trends to owners.
Cost drivers	Workflow minutes/runners, private build workers, storage, scanning, network, verification jobs and logging.
Cost-optimization techniques	Cache safely, right-size private pools, avoid duplicate rebuilds, apply cleanup policies and limit noisy logs without weakening evidence.
Operational responsibilities	Platform DevSecOps owns golden paths; product teams own code/tests and release outcomes; security owns policy; SRE owns production rollout/rollback practice.
Integration points	GitHub, IAM/WIF, Cloud Build, Artifact Registry, Cloud Deploy, Binary Authorization, Security Command Center, Monitoring, Terraform and target services.
Alternatives	GitLab/Jenkins/Azure DevOps, third-party registries/deployment tools or custom pipelines when approved requirements are unmet.
Selection criteria	Source governance, identity, network, artifact type, target support, regulated evidence, progressive delivery, operability and cost.
Best practices	Keyless identity, immutable digests, pinned dependencies, isolated builds, policy gates, same-artifact promotion, independent approval and tested rollback.
Reference architecture	GitHub -> PR checks -> WIF -> Cloud Build -> Artifact Registry -> policy/attestation -> target-specific deployment -> monitoring/rollback.
Sample configuration	See examples folders; validate exact action versions, API schema and provider arguments before production use.
Common pitfalls	Long-lived keys, mutable tags, unpinned actions, broad service accounts, rebuild per environment, unmanaged console deployment and absent rollback evidence.
Anti-patterns	CI identity is project owner; production deployment from developer laptop; secret in repository; latest tag; approval by code author only.
Production-readiness checklist	Repository controls; WIF; IAM separation; private build path; scans; immutable artifact; provenance; target approval; verification; rollback; metrics and retention.

28.3 Artifact Registry

Assessment field	Enterprise guidance
Service name	Artifact Registry
Purpose	Artifact storage and distribution
Enterprise architecture role	System of record for immutable images, packages, modules and deployment inputs
Business capabilities supported	Secure software and data delivery, controlled change, auditability, platform engineering and regulated release evidence.
Technical capabilities	Artifact storage and distribution integrated with IAM, logging, monitoring and Google Cloud resource controls.
Benefits	Managed integration, reduced undifferentiated operations, standard evidence and alignment with the inherited Google Cloud platform.
Limitations	Features, targets, quotas, locations, integrations and launch stages vary; some data assets require dedicated deployment lanes rather than application CD.
Service quotas	Validate project, regional, API, concurrency, storage and throughput quotas before scale rollout.
Regional availability considerations	Place regional resources such as private pools and repositories with the approved delivery/data plane; validate cross-region access and residency.
High-availability characteristics	Managed control planes reduce infrastructure operations; delivery workflows must tolerate API/transient failures and preserve source/artifacts for replay.
Disaster-recovery considerations	Back up source/configuration, replicate or rebuild artifact repositories where required, preserve provenance and test alternate build/deploy paths.
Security model	IAM, workload identity, protected repositories/environments, least privilege, private network access and immutable audit evidence.
IAM requirements	Separate source admin, workflow identity, build identity, artifact writer, deployer, approver and runtime identities.
Network requirements	Private pools or approved runners, Private Google Access/PSC where supported and controlled egress for dependencies.
VPC Service Controls considerations	Validate service support and ingress/egress dependencies; GitHub is outside the perimeter and must use WIF and narrowly authorized APIs.
Encryption capabilities	Google-managed encryption by default and TLS in transit; protect evidence, artifacts and configuration.
CMEK support	Use where required and supported, especially repositories/build resources; validate resource/location and service-agent permissions.
Logging	Enable audit, workflow, build, artifact and rollout logs; exclude secrets and sensitive payloads.
Monitoring	Track workflow/build/rollout success, duration, queue, policy failures, artifact age, vulnerabilities and release outcomes.
Alerting	Page only for production-impacting or supply-chain security events; route failed builds and policy trends to owners.
Cost drivers	Workflow minutes/runners, private build workers, storage, scanning, network, verification jobs and logging.
Cost-optimization techniques	Cache safely, right-size private pools, avoid duplicate rebuilds, apply cleanup policies and limit noisy logs without weakening evidence.
Operational responsibilities	Platform DevSecOps owns golden paths; product teams own code/tests and release outcomes; security owns policy; SRE owns production rollout/rollback practice.
Integration points	GitHub, IAM/WIF, Cloud Build, Artifact Registry, Cloud Deploy, Binary Authorization, Security Command Center, Monitoring, Terraform and target services.
Alternatives	GitLab/Jenkins/Azure DevOps, third-party registries/deployment tools or custom pipelines when approved requirements are unmet.
Selection criteria	Source governance, identity, network, artifact type, target support, regulated evidence, progressive delivery, operability and cost.
Best practices	Keyless identity, immutable digests, pinned dependencies, isolated builds, policy gates, same-artifact promotion, independent approval and tested rollback.
Reference architecture	GitHub -> PR checks -> WIF -> Cloud Build -> Artifact Registry -> policy/attestation -> target-specific deployment -> monitoring/rollback.
Sample configuration	See examples folders; validate exact action versions, API schema and provider arguments before production use.
Common pitfalls	Long-lived keys, mutable tags, unpinned actions, broad service accounts, rebuild per environment, unmanaged console deployment and absent rollback evidence.
Anti-patterns	CI identity is project owner; production deployment from developer laptop; secret in repository; latest tag; approval by code author only.
Production-readiness checklist	Repository controls; WIF; IAM separation; private build path; scans; immutable artifact; provenance; target approval; verification; rollback; metrics and retention.

28.4 Cloud Deploy

Assessment field	Enterprise guidance
Service name	Cloud Deploy
Purpose	Managed continuous delivery
Enterprise architecture role	Application delivery to supported targets with approval, verification and progressive rollout
Business capabilities supported	Secure software and data delivery, controlled change, auditability, platform engineering and regulated release evidence.
Technical capabilities	Managed continuous delivery integrated with IAM, logging, monitoring and Google Cloud resource controls.
Benefits	Managed integration, reduced undifferentiated operations, standard evidence and alignment with the inherited Google Cloud platform.
Limitations	Features, targets, quotas, locations, integrations and launch stages vary; some data assets require dedicated deployment lanes rather than application CD.
Service quotas	Validate project, regional, API, concurrency, storage and throughput quotas before scale rollout.
Regional availability considerations	Place regional resources such as private pools and repositories with the approved delivery/data plane; validate cross-region access and residency.
High-availability characteristics	Managed control planes reduce infrastructure operations; delivery workflows must tolerate API/transient failures and preserve source/artifacts for replay.
Disaster-recovery considerations	Back up source/configuration, replicate or rebuild artifact repositories where required, preserve provenance and test alternate build/deploy paths.
Security model	IAM, workload identity, protected repositories/environments, least privilege, private network access and immutable audit evidence.
IAM requirements	Separate source admin, workflow identity, build identity, artifact writer, deployer, approver and runtime identities.
Network requirements	Private pools or approved runners, Private Google Access/PSC where supported and controlled egress for dependencies.
VPC Service Controls considerations	Validate service support and ingress/egress dependencies; GitHub is outside the perimeter and must use WIF and narrowly authorized APIs.
Encryption capabilities	Google-managed encryption by default and TLS in transit; protect evidence, artifacts and configuration.
CMEK support	Use where required and supported, especially repositories/build resources; validate resource/location and service-agent permissions.
Logging	Enable audit, workflow, build, artifact and rollout logs; exclude secrets and sensitive payloads.
Monitoring	Track workflow/build/rollout success, duration, queue, policy failures, artifact age, vulnerabilities and release outcomes.
Alerting	Page only for production-impacting or supply-chain security events; route failed builds and policy trends to owners.
Cost drivers	Workflow minutes/runners, private build workers, storage, scanning, network, verification jobs and logging.
Cost-optimization techniques	Cache safely, right-size private pools, avoid duplicate rebuilds, apply cleanup policies and limit noisy logs without weakening evidence.
Operational responsibilities	Platform DevSecOps owns golden paths; product teams own code/tests and release outcomes; security owns policy; SRE owns production rollout/rollback practice.
Integration points	GitHub, IAM/WIF, Cloud Build, Artifact Registry, Cloud Deploy, Binary Authorization, Security Command Center, Monitoring, Terraform and target services.
Alternatives	GitLab/Jenkins/Azure DevOps, third-party registries/deployment tools or custom pipelines when approved requirements are unmet.
Selection criteria	Source governance, identity, network, artifact type, target support, regulated evidence, progressive delivery, operability and cost.
Best practices	Keyless identity, immutable digests, pinned dependencies, isolated builds, policy gates, same-artifact promotion, independent approval and tested rollback.
Reference architecture	GitHub -> PR checks -> WIF -> Cloud Build -> Artifact Registry -> policy/attestation -> target-specific deployment -> monitoring/rollback.
Sample configuration	See examples folders; validate exact action versions, API schema and provider arguments before production use.
Common pitfalls	Long-lived keys, mutable tags, unpinned actions, broad service accounts, rebuild per environment, unmanaged console deployment and absent rollback evidence.
Anti-patterns	CI identity is project owner; production deployment from developer laptop; secret in repository; latest tag; approval by code author only.
Production-readiness checklist	Repository controls; WIF; IAM separation; private build path; scans; immutable artifact; provenance; target approval; verification; rollback; metrics and retention.

28.5 Workload Identity Federation

Assessment field	Enterprise guidance
Service name	Workload Identity Federation
Purpose	Keyless external workload authentication
Enterprise architecture role	Trust bridge from GitHub and other CI systems to short-lived Google Cloud credentials
Business capabilities supported	Secure software and data delivery, controlled change, auditability, platform engineering and regulated release evidence.
Technical capabilities	Keyless external workload authentication integrated with IAM, logging, monitoring and Google Cloud resource controls.
Benefits	Managed integration, reduced undifferentiated operations, standard evidence and alignment with the inherited Google Cloud platform.
Limitations	Features, targets, quotas, locations, integrations and launch stages vary; some data assets require dedicated deployment lanes rather than application CD.
Service quotas	Validate project, regional, API, concurrency, storage and throughput quotas before scale rollout.
Regional availability considerations	Place regional resources such as private pools and repositories with the approved delivery/data plane; validate cross-region access and residency.
High-availability characteristics	Managed control planes reduce infrastructure operations; delivery workflows must tolerate API/transient failures and preserve source/artifacts for replay.
Disaster-recovery considerations	Back up source/configuration, replicate or rebuild artifact repositories where required, preserve provenance and test alternate build/deploy paths.
Security model	IAM, workload identity, protected repositories/environments, least privilege, private network access and immutable audit evidence.
IAM requirements	Separate source admin, workflow identity, build identity, artifact writer, deployer, approver and runtime identities.
Network requirements	Private pools or approved runners, Private Google Access/PSC where supported and controlled egress for dependencies.
VPC Service Controls considerations	Validate service support and ingress/egress dependencies; GitHub is outside the perimeter and must use WIF and narrowly authorized APIs.
Encryption capabilities	Google-managed encryption by default and TLS in transit; protect evidence, artifacts and configuration.
CMEK support	Use where required and supported, especially repositories/build resources; validate resource/location and service-agent permissions.
Logging	Enable audit, workflow, build, artifact and rollout logs; exclude secrets and sensitive payloads.
Monitoring	Track workflow/build/rollout success, duration, queue, policy failures, artifact age, vulnerabilities and release outcomes.
Alerting	Page only for production-impacting or supply-chain security events; route failed builds and policy trends to owners.
Cost drivers	Workflow minutes/runners, private build workers, storage, scanning, network, verification jobs and logging.
Cost-optimization techniques	Cache safely, right-size private pools, avoid duplicate rebuilds, apply cleanup policies and limit noisy logs without weakening evidence.
Operational responsibilities	Platform DevSecOps owns golden paths; product teams own code/tests and release outcomes; security owns policy; SRE owns production rollout/rollback practice.
Integration points	GitHub, IAM/WIF, Cloud Build, Artifact Registry, Cloud Deploy, Binary Authorization, Security Command Center, Monitoring, Terraform and target services.
Alternatives	GitLab/Jenkins/Azure DevOps, third-party registries/deployment tools or custom pipelines when approved requirements are unmet.
Selection criteria	Source governance, identity, network, artifact type, target support, regulated evidence, progressive delivery, operability and cost.
Best practices	Keyless identity, immutable digests, pinned dependencies, isolated builds, policy gates, same-artifact promotion, independent approval and tested rollback.
Reference architecture	GitHub -> PR checks -> WIF -> Cloud Build -> Artifact Registry -> policy/attestation -> target-specific deployment -> monitoring/rollback.
Sample configuration	See examples folders; validate exact action versions, API schema and provider arguments before production use.
Common pitfalls	Long-lived keys, mutable tags, unpinned actions, broad service accounts, rebuild per environment, unmanaged console deployment and absent rollback evidence.
Anti-patterns	CI identity is project owner; production deployment from developer laptop; secret in repository; latest tag; approval by code author only.
Production-readiness checklist	Repository controls; WIF; IAM separation; private build path; scans; immutable artifact; provenance; target approval; verification; rollback; metrics and retention.

28.6 Artifact Analysis

Assessment field	Enterprise guidance
Service name	Artifact Analysis
Purpose	Vulnerability and package analysis
Enterprise architecture role	Automated vulnerability evidence for artifacts in Artifact Registry
Business capabilities supported	Secure software and data delivery, controlled change, auditability, platform engineering and regulated release evidence.
Technical capabilities	Vulnerability and package analysis integrated with IAM, logging, monitoring and Google Cloud resource controls.
Benefits	Managed integration, reduced undifferentiated operations, standard evidence and alignment with the inherited Google Cloud platform.
Limitations	Features, targets, quotas, locations, integrations and launch stages vary; some data assets require dedicated deployment lanes rather than application CD.
Service quotas	Validate project, regional, API, concurrency, storage and throughput quotas before scale rollout.
Regional availability considerations	Place regional resources such as private pools and repositories with the approved delivery/data plane; validate cross-region access and residency.
High-availability characteristics	Managed control planes reduce infrastructure operations; delivery workflows must tolerate API/transient failures and preserve source/artifacts for replay.
Disaster-recovery considerations	Back up source/configuration, replicate or rebuild artifact repositories where required, preserve provenance and test alternate build/deploy paths.
Security model	IAM, workload identity, protected repositories/environments, least privilege, private network access and immutable audit evidence.
IAM requirements	Separate source admin, workflow identity, build identity, artifact writer, deployer, approver and runtime identities.
Network requirements	Private pools or approved runners, Private Google Access/PSC where supported and controlled egress for dependencies.
VPC Service Controls considerations	Validate service support and ingress/egress dependencies; GitHub is outside the perimeter and must use WIF and narrowly authorized APIs.
Encryption capabilities	Google-managed encryption by default and TLS in transit; protect evidence, artifacts and configuration.
CMEK support	Use where required and supported, especially repositories/build resources; validate resource/location and service-agent permissions.
Logging	Enable audit, workflow, build, artifact and rollout logs; exclude secrets and sensitive payloads.
Monitoring	Track workflow/build/rollout success, duration, queue, policy failures, artifact age, vulnerabilities and release outcomes.
Alerting	Page only for production-impacting or supply-chain security events; route failed builds and policy trends to owners.
Cost drivers	Workflow minutes/runners, private build workers, storage, scanning, network, verification jobs and logging.
Cost-optimization techniques	Cache safely, right-size private pools, avoid duplicate rebuilds, apply cleanup policies and limit noisy logs without weakening evidence.
Operational responsibilities	Platform DevSecOps owns golden paths; product teams own code/tests and release outcomes; security owns policy; SRE owns production rollout/rollback practice.
Integration points	GitHub, IAM/WIF, Cloud Build, Artifact Registry, Cloud Deploy, Binary Authorization, Security Command Center, Monitoring, Terraform and target services.
Alternatives	GitLab/Jenkins/Azure DevOps, third-party registries/deployment tools or custom pipelines when approved requirements are unmet.
Selection criteria	Source governance, identity, network, artifact type, target support, regulated evidence, progressive delivery, operability and cost.
Best practices	Keyless identity, immutable digests, pinned dependencies, isolated builds, policy gates, same-artifact promotion, independent approval and tested rollback.
Reference architecture	GitHub -> PR checks -> WIF -> Cloud Build -> Artifact Registry -> policy/attestation -> target-specific deployment -> monitoring/rollback.
Sample configuration	See examples folders; validate exact action versions, API schema and provider arguments before production use.
Common pitfalls	Long-lived keys, mutable tags, unpinned actions, broad service accounts, rebuild per environment, unmanaged console deployment and absent rollback evidence.
Anti-patterns	CI identity is project owner; production deployment from developer laptop; secret in repository; latest tag; approval by code author only.
Production-readiness checklist	Repository controls; WIF; IAM separation; private build path; scans; immutable artifact; provenance; target approval; verification; rollback; metrics and retention.

28.7 Binary Authorization

Assessment field	Enterprise guidance
Service name	Binary Authorization
Purpose	Deployment policy enforcement
Enterprise architecture role	Admission and continuous validation of container deployment policy
Business capabilities supported	Secure software and data delivery, controlled change, auditability, platform engineering and regulated release evidence.
Technical capabilities	Deployment policy enforcement integrated with IAM, logging, monitoring and Google Cloud resource controls.
Benefits	Managed integration, reduced undifferentiated operations, standard evidence and alignment with the inherited Google Cloud platform.
Limitations	Features, targets, quotas, locations, integrations and launch stages vary; some data assets require dedicated deployment lanes rather than application CD.
Service quotas	Validate project, regional, API, concurrency, storage and throughput quotas before scale rollout.
Regional availability considerations	Place regional resources such as private pools and repositories with the approved delivery/data plane; validate cross-region access and residency.
High-availability characteristics	Managed control planes reduce infrastructure operations; delivery workflows must tolerate API/transient failures and preserve source/artifacts for replay.
Disaster-recovery considerations	Back up source/configuration, replicate or rebuild artifact repositories where required, preserve provenance and test alternate build/deploy paths.
Security model	IAM, workload identity, protected repositories/environments, least privilege, private network access and immutable audit evidence.
IAM requirements	Separate source admin, workflow identity, build identity, artifact writer, deployer, approver and runtime identities.
Network requirements	Private pools or approved runners, Private Google Access/PSC where supported and controlled egress for dependencies.
VPC Service Controls considerations	Validate service support and ingress/egress dependencies; GitHub is outside the perimeter and must use WIF and narrowly authorized APIs.
Encryption capabilities	Google-managed encryption by default and TLS in transit; protect evidence, artifacts and configuration.
CMEK support	Use where required and supported, especially repositories/build resources; validate resource/location and service-agent permissions.
Logging	Enable audit, workflow, build, artifact and rollout logs; exclude secrets and sensitive payloads.
Monitoring	Track workflow/build/rollout success, duration, queue, policy failures, artifact age, vulnerabilities and release outcomes.
Alerting	Page only for production-impacting or supply-chain security events; route failed builds and policy trends to owners.
Cost drivers	Workflow minutes/runners, private build workers, storage, scanning, network, verification jobs and logging.
Cost-optimization techniques	Cache safely, right-size private pools, avoid duplicate rebuilds, apply cleanup policies and limit noisy logs without weakening evidence.
Operational responsibilities	Platform DevSecOps owns golden paths; product teams own code/tests and release outcomes; security owns policy; SRE owns production rollout/rollback practice.
Integration points	GitHub, IAM/WIF, Cloud Build, Artifact Registry, Cloud Deploy, Binary Authorization, Security Command Center, Monitoring, Terraform and target services.
Alternatives	GitLab/Jenkins/Azure DevOps, third-party registries/deployment tools or custom pipelines when approved requirements are unmet.
Selection criteria	Source governance, identity, network, artifact type, target support, regulated evidence, progressive delivery, operability and cost.
Best practices	Keyless identity, immutable digests, pinned dependencies, isolated builds, policy gates, same-artifact promotion, independent approval and tested rollback.
Reference architecture	GitHub -> PR checks -> WIF -> Cloud Build -> Artifact Registry -> policy/attestation -> target-specific deployment -> monitoring/rollback.
Sample configuration	See examples folders; validate exact action versions, API schema and provider arguments before production use.
Common pitfalls	Long-lived keys, mutable tags, unpinned actions, broad service accounts, rebuild per environment, unmanaged console deployment and absent rollback evidence.
Anti-patterns	CI identity is project owner; production deployment from developer laptop; secret in repository; latest tag; approval by code author only.
Production-readiness checklist	Repository controls; WIF; IAM separation; private build path; scans; immutable artifact; provenance; target approval; verification; rollback; metrics and retention.

28.8 Secret Manager

Assessment field	Enterprise guidance
Service name	Secret Manager
Purpose	Managed secret storage and rotation integration
Enterprise architecture role	Store unavoidable third-party credentials without embedding them in source or pipeline configuration
Business capabilities supported	Secure software and data delivery, controlled change, auditability, platform engineering and regulated release evidence.
Technical capabilities	Managed secret storage and rotation integration integrated with IAM, logging, monitoring and Google Cloud resource controls.
Benefits	Managed integration, reduced undifferentiated operations, standard evidence and alignment with the inherited Google Cloud platform.
Limitations	Features, targets, quotas, locations, integrations and launch stages vary; some data assets require dedicated deployment lanes rather than application CD.
Service quotas	Validate project, regional, API, concurrency, storage and throughput quotas before scale rollout.
Regional availability considerations	Place regional resources such as private pools and repositories with the approved delivery/data plane; validate cross-region access and residency.
High-availability characteristics	Managed control planes reduce infrastructure operations; delivery workflows must tolerate API/transient failures and preserve source/artifacts for replay.
Disaster-recovery considerations	Back up source/configuration, replicate or rebuild artifact repositories where required, preserve provenance and test alternate build/deploy paths.
Security model	IAM, workload identity, protected repositories/environments, least privilege, private network access and immutable audit evidence.
IAM requirements	Separate source admin, workflow identity, build identity, artifact writer, deployer, approver and runtime identities.
Network requirements	Private pools or approved runners, Private Google Access/PSC where supported and controlled egress for dependencies.
VPC Service Controls considerations	Validate service support and ingress/egress dependencies; GitHub is outside the perimeter and must use WIF and narrowly authorized APIs.
Encryption capabilities	Google-managed encryption by default and TLS in transit; protect evidence, artifacts and configuration.
CMEK support	Use where required and supported, especially repositories/build resources; validate resource/location and service-agent permissions.
Logging	Enable audit, workflow, build, artifact and rollout logs; exclude secrets and sensitive payloads.
Monitoring	Track workflow/build/rollout success, duration, queue, policy failures, artifact age, vulnerabilities and release outcomes.
Alerting	Page only for production-impacting or supply-chain security events; route failed builds and policy trends to owners.
Cost drivers	Workflow minutes/runners, private build workers, storage, scanning, network, verification jobs and logging.
Cost-optimization techniques	Cache safely, right-size private pools, avoid duplicate rebuilds, apply cleanup policies and limit noisy logs without weakening evidence.
Operational responsibilities	Platform DevSecOps owns golden paths; product teams own code/tests and release outcomes; security owns policy; SRE owns production rollout/rollback practice.
Integration points	GitHub, IAM/WIF, Cloud Build, Artifact Registry, Cloud Deploy, Binary Authorization, Security Command Center, Monitoring, Terraform and target services.
Alternatives	GitLab/Jenkins/Azure DevOps, third-party registries/deployment tools or custom pipelines when approved requirements are unmet.
Selection criteria	Source governance, identity, network, artifact type, target support, regulated evidence, progressive delivery, operability and cost.
Best practices	Keyless identity, immutable digests, pinned dependencies, isolated builds, policy gates, same-artifact promotion, independent approval and tested rollback.
Reference architecture	GitHub -> PR checks -> WIF -> Cloud Build -> Artifact Registry -> policy/attestation -> target-specific deployment -> monitoring/rollback.
Sample configuration	See examples folders; validate exact action versions, API schema and provider arguments before production use.
Common pitfalls	Long-lived keys, mutable tags, unpinned actions, broad service accounts, rebuild per environment, unmanaged console deployment and absent rollback evidence.
Anti-patterns	CI identity is project owner; production deployment from developer laptop; secret in repository; latest tag; approval by code author only.
Production-readiness checklist	Repository controls; WIF; IAM separation; private build path; scans; immutable artifact; provenance; target approval; verification; rollback; metrics and retention.

29. Architecture Decision Records

Ten full ADR files are included in the package and summarized in this document.

ADR	Decision title	Status
ADR-0901	Use GitHub Enterprise as the primary source-control platform	Approved reference
ADR-0902	Use a federated multi-repository strategy	Approved reference
ADR-0903	Use trunk-based development with short-lived branches	Approved reference
ADR-0904	Use semantic versioning and immutable release manifests	Approved reference
ADR-0905	Split orchestration between GitHub Actions and Cloud Build	Approved reference
ADR-0906	Use Workload Identity Federation for external CI/CD	Approved reference
ADR-0907	Use Artifact Registry as the enterprise artifact system of record	Approved reference
ADR-0908	Use Cloud Deploy for supported application continuous delivery	Approved reference
ADR-0909	Enforce policy-as-code and supply-chain attestations	Approved reference
ADR-0910	Target SLSA-aligned provenance and progressive deployment	Approved reference

ADR-0901 - Use GitHub Enterprise as the primary source-control platform

Field	Decision record
ADR ID	ADR-0901
Decision title	Use GitHub Enterprise as the primary source-control platform
Status	Approved reference decision
Date	12 July 2026
Architecture domain	DevSecOps and CI/CD
Problem	The enterprise requires secure, repeatable and auditable release processes across infrastructure, applications, data pipelines and analytical assets.
Context	The platform spans AMER, EMEA and APAC, regulated data, federated product teams and multiple Google Cloud services. The delivery system must preserve separation of duties, artifact integrity, environment isolation, SLOs and recovery.
Decision drivers	Developer velocity, supply-chain integrity, least privilege, reproducibility, auditability, deployment safety, recovery and cost efficiency.
Constraints	GitHub and Google Cloud product capabilities, regional/private connectivity, third-party actions, service-specific deployment semantics and regulatory approval requirements.
Assumptions	GitHub Enterprise, Cloud Build, Artifact Registry, Cloud Deploy, WIF, Terraform and the security/operations baselines from Volumes 8, 10 and 11 are available.
Alternatives considered	Single tool for every asset; manual/console release; long-lived credentials; mutable artifacts; bespoke per-team pipelines.
Evaluation criteria	Security, provenance, supported targets, developer experience, network reach, audit evidence, operational effort, portability and economics.
Decision	Adopt GitHub Enterprise organizations with SSO, repository rulesets, CODEOWNERS, protected environments and enterprise audit integration.
Rationale	The selected approach separates source orchestration, trusted build and target-specific delivery while promoting one immutable artifact and preserving evidence.
Benefits	Consistent controls, keyless authentication, faster feedback, repeatable releases, safer rollback and stronger audit evidence.
Tradeoffs	Multiple integrated services, golden workflow maintenance, policy tuning and skills are required.
Positive consequences	Teams consume paved roads and produce standard release evidence.
Negative consequences	Non-standard tools or assets require adapters and exception review.
Risks	Provider/action compromise, policy bypass, credential misconfiguration, unsupported target semantics or failed rollback.
Mitigations	Pinned dependencies, WIF conditions, private builds, attestations, staged policies, independent approvals, canary and exercises.
Implementation implications	Publish repository standards, golden workflows, IAM modules, artifact policies, release manifest, test stages and service-specific rollback runbooks.
Cost implications	Build minutes/private pools, artifact storage/scanning, deployment verification and engineering support; measure per release and product.
Security implications	Eliminates keys, improves provenance and supports admission enforcement, but requires strict identity conditions and supply-chain governance.
Operational implications	Release SLOs, deployment telemetry, on-call, rollback, change records and vendor escalation must be operational.
Review triggers	Major product change, supply-chain incident, new regulated workload, repeated release failure, new target or SLSA revision.
Review date	12 January 2027 or earlier trigger.

ADR-0902 - Use a federated multi-repository strategy

Field	Decision record
ADR ID	ADR-0902
Decision title	Use a federated multi-repository strategy
Status	Approved reference decision
Date	12 July 2026
Architecture domain	DevSecOps and CI/CD
Problem	The enterprise requires secure, repeatable and auditable release processes across infrastructure, applications, data pipelines and analytical assets.
Context	The platform spans AMER, EMEA and APAC, regulated data, federated product teams and multiple Google Cloud services. The delivery system must preserve separation of duties, artifact integrity, environment isolation, SLOs and recovery.
Decision drivers	Developer velocity, supply-chain integrity, least privilege, reproducibility, auditability, deployment safety, recovery and cost efficiency.
Constraints	GitHub and Google Cloud product capabilities, regional/private connectivity, third-party actions, service-specific deployment semantics and regulatory approval requirements.
Assumptions	GitHub Enterprise, Cloud Build, Artifact Registry, Cloud Deploy, WIF, Terraform and the security/operations baselines from Volumes 8, 10 and 11 are available.
Alternatives considered	Single tool for every asset; manual/console release; long-lived credentials; mutable artifacts; bespoke per-team pipelines.
Evaluation criteria	Security, provenance, supported targets, developer experience, network reach, audit evidence, operational effort, portability and economics.
Decision	Use separate repositories for platform modules, policy libraries, domain data products and applications; allow a monorepo only when one ownership and release boundary is demonstrably stronger.
Rationale	The selected approach separates source orchestration, trusted build and target-specific delivery while promoting one immutable artifact and preserving evidence.
Benefits	Consistent controls, keyless authentication, faster feedback, repeatable releases, safer rollback and stronger audit evidence.
Tradeoffs	Multiple integrated services, golden workflow maintenance, policy tuning and skills are required.
Positive consequences	Teams consume paved roads and produce standard release evidence.
Negative consequences	Non-standard tools or assets require adapters and exception review.
Risks	Provider/action compromise, policy bypass, credential misconfiguration, unsupported target semantics or failed rollback.
Mitigations	Pinned dependencies, WIF conditions, private builds, attestations, staged policies, independent approvals, canary and exercises.
Implementation implications	Publish repository standards, golden workflows, IAM modules, artifact policies, release manifest, test stages and service-specific rollback runbooks.
Cost implications	Build minutes/private pools, artifact storage/scanning, deployment verification and engineering support; measure per release and product.
Security implications	Eliminates keys, improves provenance and supports admission enforcement, but requires strict identity conditions and supply-chain governance.
Operational implications	Release SLOs, deployment telemetry, on-call, rollback, change records and vendor escalation must be operational.
Review triggers	Major product change, supply-chain incident, new regulated workload, repeated release failure, new target or SLSA revision.
Review date	12 January 2027 or earlier trigger.

ADR-0903 - Use trunk-based development with short-lived branches

Field	Decision record
ADR ID	ADR-0903
Decision title	Use trunk-based development with short-lived branches
Status	Approved reference decision
Date	12 July 2026
Architecture domain	DevSecOps and CI/CD
Problem	The enterprise requires secure, repeatable and auditable release processes across infrastructure, applications, data pipelines and analytical assets.
Context	The platform spans AMER, EMEA and APAC, regulated data, federated product teams and multiple Google Cloud services. The delivery system must preserve separation of duties, artifact integrity, environment isolation, SLOs and recovery.
Decision drivers	Developer velocity, supply-chain integrity, least privilege, reproducibility, auditability, deployment safety, recovery and cost efficiency.
Constraints	GitHub and Google Cloud product capabilities, regional/private connectivity, third-party actions, service-specific deployment semantics and regulatory approval requirements.
Assumptions	GitHub Enterprise, Cloud Build, Artifact Registry, Cloud Deploy, WIF, Terraform and the security/operations baselines from Volumes 8, 10 and 11 are available.
Alternatives considered	Single tool for every asset; manual/console release; long-lived credentials; mutable artifacts; bespoke per-team pipelines.
Evaluation criteria	Security, provenance, supported targets, developer experience, network reach, audit evidence, operational effort, portability and economics.
Decision	Merge through protected pull requests to main; use release branches only for supported maintenance or tightly controlled regulated releases.
Rationale	The selected approach separates source orchestration, trusted build and target-specific delivery while promoting one immutable artifact and preserving evidence.
Benefits	Consistent controls, keyless authentication, faster feedback, repeatable releases, safer rollback and stronger audit evidence.
Tradeoffs	Multiple integrated services, golden workflow maintenance, policy tuning and skills are required.
Positive consequences	Teams consume paved roads and produce standard release evidence.
Negative consequences	Non-standard tools or assets require adapters and exception review.
Risks	Provider/action compromise, policy bypass, credential misconfiguration, unsupported target semantics or failed rollback.
Mitigations	Pinned dependencies, WIF conditions, private builds, attestations, staged policies, independent approvals, canary and exercises.
Implementation implications	Publish repository standards, golden workflows, IAM modules, artifact policies, release manifest, test stages and service-specific rollback runbooks.
Cost implications	Build minutes/private pools, artifact storage/scanning, deployment verification and engineering support; measure per release and product.
Security implications	Eliminates keys, improves provenance and supports admission enforcement, but requires strict identity conditions and supply-chain governance.
Operational implications	Release SLOs, deployment telemetry, on-call, rollback, change records and vendor escalation must be operational.
Review triggers	Major product change, supply-chain incident, new regulated workload, repeated release failure, new target or SLSA revision.
Review date	12 January 2027 or earlier trigger.

ADR-0904 - Use semantic versioning and immutable release manifests

Field	Decision record
ADR ID	ADR-0904
Decision title	Use semantic versioning and immutable release manifests
Status	Approved reference decision
Date	12 July 2026
Architecture domain	DevSecOps and CI/CD
Problem	The enterprise requires secure, repeatable and auditable release processes across infrastructure, applications, data pipelines and analytical assets.
Context	The platform spans AMER, EMEA and APAC, regulated data, federated product teams and multiple Google Cloud services. The delivery system must preserve separation of duties, artifact integrity, environment isolation, SLOs and recovery.
Decision drivers	Developer velocity, supply-chain integrity, least privilege, reproducibility, auditability, deployment safety, recovery and cost efficiency.
Constraints	GitHub and Google Cloud product capabilities, regional/private connectivity, third-party actions, service-specific deployment semantics and regulatory approval requirements.
Assumptions	GitHub Enterprise, Cloud Build, Artifact Registry, Cloud Deploy, WIF, Terraform and the security/operations baselines from Volumes 8, 10 and 11 are available.
Alternatives considered	Single tool for every asset; manual/console release; long-lived credentials; mutable artifacts; bespoke per-team pipelines.
Evaluation criteria	Security, provenance, supported targets, developer experience, network reach, audit evidence, operational effort, portability and economics.
Decision	Version reusable modules and products semantically; identify runtime artifacts by digest and record a release manifest linking all evidence.
Rationale	The selected approach separates source orchestration, trusted build and target-specific delivery while promoting one immutable artifact and preserving evidence.
Benefits	Consistent controls, keyless authentication, faster feedback, repeatable releases, safer rollback and stronger audit evidence.
Tradeoffs	Multiple integrated services, golden workflow maintenance, policy tuning and skills are required.
Positive consequences	Teams consume paved roads and produce standard release evidence.
Negative consequences	Non-standard tools or assets require adapters and exception review.
Risks	Provider/action compromise, policy bypass, credential misconfiguration, unsupported target semantics or failed rollback.
Mitigations	Pinned dependencies, WIF conditions, private builds, attestations, staged policies, independent approvals, canary and exercises.
Implementation implications	Publish repository standards, golden workflows, IAM modules, artifact policies, release manifest, test stages and service-specific rollback runbooks.
Cost implications	Build minutes/private pools, artifact storage/scanning, deployment verification and engineering support; measure per release and product.
Security implications	Eliminates keys, improves provenance and supports admission enforcement, but requires strict identity conditions and supply-chain governance.
Operational implications	Release SLOs, deployment telemetry, on-call, rollback, change records and vendor escalation must be operational.
Review triggers	Major product change, supply-chain incident, new regulated workload, repeated release failure, new target or SLSA revision.
Review date	12 January 2027 or earlier trigger.

ADR-0905 - Split orchestration between GitHub Actions and Cloud Build

Field	Decision record
ADR ID	ADR-0905
Decision title	Split orchestration between GitHub Actions and Cloud Build
Status	Approved reference decision
Date	12 July 2026
Architecture domain	DevSecOps and CI/CD
Problem	The enterprise requires secure, repeatable and auditable release processes across infrastructure, applications, data pipelines and analytical assets.
Context	The platform spans AMER, EMEA and APAC, regulated data, federated product teams and multiple Google Cloud services. The delivery system must preserve separation of duties, artifact integrity, environment isolation, SLOs and recovery.
Decision drivers	Developer velocity, supply-chain integrity, least privilege, reproducibility, auditability, deployment safety, recovery and cost efficiency.
Constraints	GitHub and Google Cloud product capabilities, regional/private connectivity, third-party actions, service-specific deployment semantics and regulatory approval requirements.
Assumptions	GitHub Enterprise, Cloud Build, Artifact Registry, Cloud Deploy, WIF, Terraform and the security/operations baselines from Volumes 8, 10 and 11 are available.
Alternatives considered	Single tool for every asset; manual/console release; long-lived credentials; mutable artifacts; bespoke per-team pipelines.
Evaluation criteria	Security, provenance, supported targets, developer experience, network reach, audit evidence, operational effort, portability and economics.
Decision	Use GitHub Actions for repository-native orchestration and Cloud Build for trusted Google Cloud builds, private network access, provenance and artifact publication.
Rationale	The selected approach separates source orchestration, trusted build and target-specific delivery while promoting one immutable artifact and preserving evidence.
Benefits	Consistent controls, keyless authentication, faster feedback, repeatable releases, safer rollback and stronger audit evidence.
Tradeoffs	Multiple integrated services, golden workflow maintenance, policy tuning and skills are required.
Positive consequences	Teams consume paved roads and produce standard release evidence.
Negative consequences	Non-standard tools or assets require adapters and exception review.
Risks	Provider/action compromise, policy bypass, credential misconfiguration, unsupported target semantics or failed rollback.
Mitigations	Pinned dependencies, WIF conditions, private builds, attestations, staged policies, independent approvals, canary and exercises.
Implementation implications	Publish repository standards, golden workflows, IAM modules, artifact policies, release manifest, test stages and service-specific rollback runbooks.
Cost implications	Build minutes/private pools, artifact storage/scanning, deployment verification and engineering support; measure per release and product.
Security implications	Eliminates keys, improves provenance and supports admission enforcement, but requires strict identity conditions and supply-chain governance.
Operational implications	Release SLOs, deployment telemetry, on-call, rollback, change records and vendor escalation must be operational.
Review triggers	Major product change, supply-chain incident, new regulated workload, repeated release failure, new target or SLSA revision.
Review date	12 January 2027 or earlier trigger.

ADR-0906 - Use Workload Identity Federation for external CI/CD

Field	Decision record
ADR ID	ADR-0906
Decision title	Use Workload Identity Federation for external CI/CD
Status	Approved reference decision
Date	12 July 2026
Architecture domain	DevSecOps and CI/CD
Problem	The enterprise requires secure, repeatable and auditable release processes across infrastructure, applications, data pipelines and analytical assets.
Context	The platform spans AMER, EMEA and APAC, regulated data, federated product teams and multiple Google Cloud services. The delivery system must preserve separation of duties, artifact integrity, environment isolation, SLOs and recovery.
Decision drivers	Developer velocity, supply-chain integrity, least privilege, reproducibility, auditability, deployment safety, recovery and cost efficiency.
Constraints	GitHub and Google Cloud product capabilities, regional/private connectivity, third-party actions, service-specific deployment semantics and regulatory approval requirements.
Assumptions	GitHub Enterprise, Cloud Build, Artifact Registry, Cloud Deploy, WIF, Terraform and the security/operations baselines from Volumes 8, 10 and 11 are available.
Alternatives considered	Single tool for every asset; manual/console release; long-lived credentials; mutable artifacts; bespoke per-team pipelines.
Evaluation criteria	Security, provenance, supported targets, developer experience, network reach, audit evidence, operational effort, portability and economics.
Decision	Use OIDC federation and service-account impersonation with repository/ref/workflow/environment conditions; prohibit service-account keys.
Rationale	The selected approach separates source orchestration, trusted build and target-specific delivery while promoting one immutable artifact and preserving evidence.
Benefits	Consistent controls, keyless authentication, faster feedback, repeatable releases, safer rollback and stronger audit evidence.
Tradeoffs	Multiple integrated services, golden workflow maintenance, policy tuning and skills are required.
Positive consequences	Teams consume paved roads and produce standard release evidence.
Negative consequences	Non-standard tools or assets require adapters and exception review.
Risks	Provider/action compromise, policy bypass, credential misconfiguration, unsupported target semantics or failed rollback.
Mitigations	Pinned dependencies, WIF conditions, private builds, attestations, staged policies, independent approvals, canary and exercises.
Implementation implications	Publish repository standards, golden workflows, IAM modules, artifact policies, release manifest, test stages and service-specific rollback runbooks.
Cost implications	Build minutes/private pools, artifact storage/scanning, deployment verification and engineering support; measure per release and product.
Security implications	Eliminates keys, improves provenance and supports admission enforcement, but requires strict identity conditions and supply-chain governance.
Operational implications	Release SLOs, deployment telemetry, on-call, rollback, change records and vendor escalation must be operational.
Review triggers	Major product change, supply-chain incident, new regulated workload, repeated release failure, new target or SLSA revision.
Review date	12 January 2027 or earlier trigger.

ADR-0907 - Use Artifact Registry as the enterprise artifact system of record

Field	Decision record
ADR ID	ADR-0907
Decision title	Use Artifact Registry as the enterprise artifact system of record
Status	Approved reference decision
Date	12 July 2026
Architecture domain	DevSecOps and CI/CD
Problem	The enterprise requires secure, repeatable and auditable release processes across infrastructure, applications, data pipelines and analytical assets.
Context	The platform spans AMER, EMEA and APAC, regulated data, federated product teams and multiple Google Cloud services. The delivery system must preserve separation of duties, artifact integrity, environment isolation, SLOs and recovery.
Decision drivers	Developer velocity, supply-chain integrity, least privilege, reproducibility, auditability, deployment safety, recovery and cost efficiency.
Constraints	GitHub and Google Cloud product capabilities, regional/private connectivity, third-party actions, service-specific deployment semantics and regulatory approval requirements.
Assumptions	GitHub Enterprise, Cloud Build, Artifact Registry, Cloud Deploy, WIF, Terraform and the security/operations baselines from Volumes 8, 10 and 11 are available.
Alternatives considered	Single tool for every asset; manual/console release; long-lived credentials; mutable artifacts; bespoke per-team pipelines.
Evaluation criteria	Security, provenance, supported targets, developer experience, network reach, audit evidence, operational effort, portability and economics.
Decision	Store images, packages and modules in regional repositories with immutable tags/digests, scanning, CMEK where required and controlled cleanup.
Rationale	The selected approach separates source orchestration, trusted build and target-specific delivery while promoting one immutable artifact and preserving evidence.
Benefits	Consistent controls, keyless authentication, faster feedback, repeatable releases, safer rollback and stronger audit evidence.
Tradeoffs	Multiple integrated services, golden workflow maintenance, policy tuning and skills are required.
Positive consequences	Teams consume paved roads and produce standard release evidence.
Negative consequences	Non-standard tools or assets require adapters and exception review.
Risks	Provider/action compromise, policy bypass, credential misconfiguration, unsupported target semantics or failed rollback.
Mitigations	Pinned dependencies, WIF conditions, private builds, attestations, staged policies, independent approvals, canary and exercises.
Implementation implications	Publish repository standards, golden workflows, IAM modules, artifact policies, release manifest, test stages and service-specific rollback runbooks.
Cost implications	Build minutes/private pools, artifact storage/scanning, deployment verification and engineering support; measure per release and product.
Security implications	Eliminates keys, improves provenance and supports admission enforcement, but requires strict identity conditions and supply-chain governance.
Operational implications	Release SLOs, deployment telemetry, on-call, rollback, change records and vendor escalation must be operational.
Review triggers	Major product change, supply-chain incident, new regulated workload, repeated release failure, new target or SLSA revision.
Review date	12 January 2027 or earlier trigger.

ADR-0908 - Use Cloud Deploy for supported application continuous delivery

Field	Decision record
ADR ID	ADR-0908
Decision title	Use Cloud Deploy for supported application continuous delivery
Status	Approved reference decision
Date	12 July 2026
Architecture domain	DevSecOps and CI/CD
Problem	The enterprise requires secure, repeatable and auditable release processes across infrastructure, applications, data pipelines and analytical assets.
Context	The platform spans AMER, EMEA and APAC, regulated data, federated product teams and multiple Google Cloud services. The delivery system must preserve separation of duties, artifact integrity, environment isolation, SLOs and recovery.
Decision drivers	Developer velocity, supply-chain integrity, least privilege, reproducibility, auditability, deployment safety, recovery and cost efficiency.
Constraints	GitHub and Google Cloud product capabilities, regional/private connectivity, third-party actions, service-specific deployment semantics and regulatory approval requirements.
Assumptions	GitHub Enterprise, Cloud Build, Artifact Registry, Cloud Deploy, WIF, Terraform and the security/operations baselines from Volumes 8, 10 and 11 are available.
Alternatives considered	Single tool for every asset; manual/console release; long-lived credentials; mutable artifacts; bespoke per-team pipelines.
Evaluation criteria	Security, provenance, supported targets, developer experience, network reach, audit evidence, operational effort, portability and economics.
Decision	Use Cloud Deploy for Cloud Run/GKE and supported targets with approval, verification and canary/standard strategies; use dedicated pipelines for Terraform and data assets.
Rationale	The selected approach separates source orchestration, trusted build and target-specific delivery while promoting one immutable artifact and preserving evidence.
Benefits	Consistent controls, keyless authentication, faster feedback, repeatable releases, safer rollback and stronger audit evidence.
Tradeoffs	Multiple integrated services, golden workflow maintenance, policy tuning and skills are required.
Positive consequences	Teams consume paved roads and produce standard release evidence.
Negative consequences	Non-standard tools or assets require adapters and exception review.
Risks	Provider/action compromise, policy bypass, credential misconfiguration, unsupported target semantics or failed rollback.
Mitigations	Pinned dependencies, WIF conditions, private builds, attestations, staged policies, independent approvals, canary and exercises.
Implementation implications	Publish repository standards, golden workflows, IAM modules, artifact policies, release manifest, test stages and service-specific rollback runbooks.
Cost implications	Build minutes/private pools, artifact storage/scanning, deployment verification and engineering support; measure per release and product.
Security implications	Eliminates keys, improves provenance and supports admission enforcement, but requires strict identity conditions and supply-chain governance.
Operational implications	Release SLOs, deployment telemetry, on-call, rollback, change records and vendor escalation must be operational.
Review triggers	Major product change, supply-chain incident, new regulated workload, repeated release failure, new target or SLSA revision.
Review date	12 January 2027 or earlier trigger.

ADR-0909 - Enforce policy-as-code and supply-chain attestations

Field	Decision record
ADR ID	ADR-0909
Decision title	Enforce policy-as-code and supply-chain attestations
Status	Approved reference decision
Date	12 July 2026
Architecture domain	DevSecOps and CI/CD
Problem	The enterprise requires secure, repeatable and auditable release processes across infrastructure, applications, data pipelines and analytical assets.
Context	The platform spans AMER, EMEA and APAC, regulated data, federated product teams and multiple Google Cloud services. The delivery system must preserve separation of duties, artifact integrity, environment isolation, SLOs and recovery.
Decision drivers	Developer velocity, supply-chain integrity, least privilege, reproducibility, auditability, deployment safety, recovery and cost efficiency.
Constraints	GitHub and Google Cloud product capabilities, regional/private connectivity, third-party actions, service-specific deployment semantics and regulatory approval requirements.
Assumptions	GitHub Enterprise, Cloud Build, Artifact Registry, Cloud Deploy, WIF, Terraform and the security/operations baselines from Volumes 8, 10 and 11 are available.
Alternatives considered	Single tool for every asset; manual/console release; long-lived credentials; mutable artifacts; bespoke per-team pipelines.
Evaluation criteria	Security, provenance, supported targets, developer experience, network reach, audit evidence, operational effort, portability and economics.
Decision	Use OPA/confest plus Artifact Analysis and Binary Authorization attestations; stage policies through audit/dry-run before enforcement.
Rationale	The selected approach separates source orchestration, trusted build and target-specific delivery while promoting one immutable artifact and preserving evidence.
Benefits	Consistent controls, keyless authentication, faster feedback, repeatable releases, safer rollback and stronger audit evidence.
Tradeoffs	Multiple integrated services, golden workflow maintenance, policy tuning and skills are required.
Positive consequences	Teams consume paved roads and produce standard release evidence.
Negative consequences	Non-standard tools or assets require adapters and exception review.
Risks	Provider/action compromise, policy bypass, credential misconfiguration, unsupported target semantics or failed rollback.
Mitigations	Pinned dependencies, WIF conditions, private builds, attestations, staged policies, independent approvals, canary and exercises.
Implementation implications	Publish repository standards, golden workflows, IAM modules, artifact policies, release manifest, test stages and service-specific rollback runbooks.
Cost implications	Build minutes/private pools, artifact storage/scanning, deployment verification and engineering support; measure per release and product.
Security implications	Eliminates keys, improves provenance and supports admission enforcement, but requires strict identity conditions and supply-chain governance.
Operational implications	Release SLOs, deployment telemetry, on-call, rollback, change records and vendor escalation must be operational.
Review triggers	Major product change, supply-chain incident, new regulated workload, repeated release failure, new target or SLSA revision.
Review date	12 January 2027 or earlier trigger.

ADR-0910 - Target SLSA-aligned provenance and progressive deployment

Field	Decision record
ADR ID	ADR-0910
Decision title	Target SLSA-aligned provenance and progressive deployment
Status	Approved reference decision
Date	12 July 2026
Architecture domain	DevSecOps and CI/CD
Problem	The enterprise requires secure, repeatable and auditable release processes across infrastructure, applications, data pipelines and analytical assets.
Context	The platform spans AMER, EMEA and APAC, regulated data, federated product teams and multiple Google Cloud services. The delivery system must preserve separation of duties, artifact integrity, environment isolation, SLOs and recovery.
Decision drivers	Developer velocity, supply-chain integrity, least privilege, reproducibility, auditability, deployment safety, recovery and cost efficiency.
Constraints	GitHub and Google Cloud product capabilities, regional/private connectivity, third-party actions, service-specific deployment semantics and regulatory approval requirements.
Assumptions	GitHub Enterprise, Cloud Build, Artifact Registry, Cloud Deploy, WIF, Terraform and the security/operations baselines from Volumes 8, 10 and 11 are available.
Alternatives considered	Single tool for every asset; manual/console release; long-lived credentials; mutable artifacts; bespoke per-team pipelines.
Evaluation criteria	Security, provenance, supported targets, developer experience, network reach, audit evidence, operational effort, portability and economics.
Decision	Generate and validate provenance, isolate trusted builds, promote immutable artifacts and use canary/blue-green or standard rollout based on workload risk.
Rationale	The selected approach separates source orchestration, trusted build and target-specific delivery while promoting one immutable artifact and preserving evidence.
Benefits	Consistent controls, keyless authentication, faster feedback, repeatable releases, safer rollback and stronger audit evidence.
Tradeoffs	Multiple integrated services, golden workflow maintenance, policy tuning and skills are required.
Positive consequences	Teams consume paved roads and produce standard release evidence.
Negative consequences	Non-standard tools or assets require adapters and exception review.
Risks	Provider/action compromise, policy bypass, credential misconfiguration, unsupported target semantics or failed rollback.
Mitigations	Pinned dependencies, WIF conditions, private builds, attestations, staged policies, independent approvals, canary and exercises.
Implementation implications	Publish repository standards, golden workflows, IAM modules, artifact policies, release manifest, test stages and service-specific rollback runbooks.
Cost implications	Build minutes/private pools, artifact storage/scanning, deployment verification and engineering support; measure per release and product.
Security implications	Eliminates keys, improves provenance and supports admission enforcement, but requires strict identity conditions and supply-chain governance.
Operational implications	Release SLOs, deployment telemetry, on-call, rollback, change records and vendor escalation must be operational.
Review triggers	Major product change, supply-chain incident, new regulated workload, repeated release failure, new target or SLSA revision.
Review date	12 January 2027 or earlier trigger.

30. Risks and Mitigations

ID	Risk	Likelihood	Impact	Mitigation	Owner
R-0901	Compromised repository workflow impersonates production identity	Low	Critical	WIF attribute conditions, protected workflows/environments, short-lived tokens and audit	IAM/DevSecOps
R-0902	Mutable tag causes untested image deployment	Medium	High	Digest pinning and immutable tags	DevSecOps
R-0903	Third-party action or builder is compromised	Medium	High	Pin by digest/SHA, allowlist, provenance and review	Security
R-0904	Terraform apply differs from reviewed plan	Medium	Critical	saved plan, state serial check, short approval window and apply identity separation	Platform
R-0905	Schema migration cannot be rolled back	Medium	High	expand/contract, compatibility views, snapshot and forward-fix plan	Data owner
R-0906	Dataflow update corrupts state or duplicates outputs	Low	Critical	compatibility test, snapshot/drain/replay and reconciliation	Data engineering
R-0907	Policy gate produces excessive false positives and bypass culture	High	Medium	policy tests, severity tiers, exception expiry and developer feedback	Security
R-0908	Build pool can exfiltrate source or secrets	Low	Critical	private pool, controlled egress, no long-lived secrets and logging	Security
R-0909	Canary uses technical health only and misses data/business regression	Medium	High	business KPI and data-quality verification	Product/SRE
R-0910	Artifact cleanup deletes rollback release	Low	High	keep policy for approved releases and recovery test	Artifact owner

31. Implementation Roadmap

Phase	Duration	Scope	Exit criteria
0 - Mobilize	2-4 weeks	inventory repos/pipelines, define standards, owners, risk classes and pilots	approved target architecture and backlog
1 - Source and identity foundation	4-8 weeks	GitHub enterprise controls, WIF, groups and audit integration	keyless pilot repository and denied-claim tests
2 - Trusted build and artifacts	6-10 weeks	Cloud Build private pools, Artifact Registry, provenance, scans and cleanup	immutable artifact with evidence
3 - Delivery lanes	8-12 weeks	Cloud Deploy, Terraform, Dataform, Dataflow, Airflow and BigQuery pipelines	representative production release and rollback
4 - Policy and supply chain	6-10 weeks	OPA, Binary Authorization, attestations and continuous validation	mandatory policy enforced after dry-run
5 - Scale and optimize	ongoing	domain onboarding, metrics, cost, policy tuning and legacy retirement	>=90% new production releases on paved roads

32. Validation and Production Readiness

Validation area	Acceptance criterion
Scope	All 48 Volume 9 topics are addressed
Identity	No long-lived Google Cloud keys in external pipelines
Repositories	rulesets, CODEOWNERS, environments and audit are enabled
Build	trusted build produces immutable artifact, provenance and test/scan evidence
Promotion	same digest/commit-derived release reaches production
Policy	mandatory security/IaC/artifact gates fail closed and have tested exceptions
Deployment	service-specific verification, canary/blue-green/standard strategy and rollback are tested
Data assets	schema, Dataform, Dataflow and Airflow lanes include data-quality/state recovery controls
Audit	commit-to-production evidence chain is queryable
Packaging	DOCX/PDF, examples, diagrams, matrices, ADRs, manifest and checksums validate

ID	Control	Classification	Evidence
PR-001	Repository owner, CODEOWNERS and branch/ruleset controls approved	Mandatory	repository settings export
PR-002	Pipeline authentication uses WIF or managed identity; no Google Cloud keys	Mandatory	IAM/WIF evidence
PR-003	Build, deploy and runtime identities are separate and least privileged	Mandatory	IAM matrix
PR-004	Artifact is immutable, digest-addressed and has provenance/test evidence	Mandatory	release manifest
PR-005	Secret, dependency, container, IaC and policy scans pass	Mandatory	scan reports
PR-006	Environment promotion uses same artifact and controlled configuration	Mandatory	digest comparison
PR-007	Production approval is independent and risk-based	Mandatory	approval record
PR-008	Deployment verification includes SLO, business and data-quality checks	Mandatory	verification job
PR-009	Rollback or forward-fix path is service-specific and tested	Mandatory	exercise evidence
PR-010	Release annotations, audit logs, metrics and owner labels are available	Mandatory	dashboard/audit
PR-011	Emergency change path uses PAM/break-glass and post-review	Mandatory	runbook/test
PR-012	Artifact retention and cleanup preserve approved rollback releases	Mandatory	repository policy

33. Dependencies, Decisions and Next Actions

Dependency	Required interface
Volume 4	Pipeline artifact, state/update, data-quality, replay and rollback requirements
Volume 5	Product ownership, contracts, certification and domain approval
Volume 7	MLOps/LLMOps prompt, model, index, agent and evaluation release bundles
Volume 8	WIF, supply-chain, Binary Authorization, secrets, audit and incident controls
Volume 10	CI/CD cost allocation and optimization
Volume 11	release annotations, deployment monitoring, SLOs and incident/PIR
Volume 12	source/artifact/configuration recovery and alternate delivery paths
Volume 13	golden Terraform modules, state and apply pipelines

1. Approve the ten Volume 9 ADRs and repository/identity model.
2. Pilot one application, Terraform root, Dataform product and Dataflow streaming release lane.
3. Create GitHub organization rulesets, WIF providers and private Cloud Build pools in non-production.
4. Publish release manifest, policy library, test-stage and approval standards.
5. Enable immutable Artifact Registry repositories, scanning and retention controls.
6. Exercise canary, application rollback, Terraform recovery, schema forward-fix and Dataflow drain/replay.
7. Begin cross-volume validation after Volume 13 completion.

34. References

Reference	Official location
Cloud Build private pools	https://docs.cloud.google.com/build/docs/private-pools/private-pools-overview
Cloud Build provenance	https://docs.cloud.google.com/build/docs/securing-builds/generate-validate-build-provenance
Workload Identity Federation for deployment pipelines	https://docs.cloud.google.com/iam/docs/workload-identity-federation-with-deployment-pipelines
Service-account best practices for deployment pipelines	https://docs.cloud.google.com/iam/docs/best-practices-for-using-service-accounts-in-deployment-pipelines
Artifact Registry analysis	https://docs.cloud.google.com/artifact-registry/docs/analysis
Artifact Registry repository settings	https://docs.cloud.google.com/artifact-registry/docs/repositories/update-repo-settings
Artifact Registry cleanup policies	https://docs.cloud.google.com/artifact-registry/docs/repositories/cleanup-policy
Cloud Deploy overview	https://docs.cloud.google.com/deploy/docs/overview
Cloud Deploy deployment strategies	https://docs.cloud.google.com/deploy/docs/deployment-strategies
Cloud Deploy verification	https://docs.cloud.google.com/deploy/docs/verify-deployment
Cloud Deploy metrics	https://docs.cloud.google.com/deploy/docs/metrics
Binary Authorization overview	https://docs.cloud.google.com/binary-authorization/docs/overview
Binary Authorization continuous validation	https://docs.cloud.google.com/binary-authorization/docs/overview-cv
SLSA specification	https://slsa.dev/spec/v1.2/

Validation note - Official documentation was checked for the July 2026 baseline. Exact feature support, API schema, quotas, regions, pricing and release stages must be revalidated during implementation.